

Building the C++ Matching Engine

Building Low Latency Applications with C++

Central Thesis

Low latency = memory layout + deterministic control flow

Topics

1. System Architecture

2. Lock-Free Queues

3. Order Book Structures

4. Matching Algorithm

5. Hot-Path Analysis

6. Design Trade-offs

Low latency = memory layout + deterministic control flow



Lock-Free Queues

Avoid lock contention between threads → deterministic inter-thread latency



Array-Backed Maps

Direct index access ($\text{price} \% N$)
→ $O(1)$ with no pointer chasing



Intrusive Linked Lists

$O(1)$ unlink — pointer already in the node, no separate list-node allocation



Memory Pools

No heap allocation in the hot path → no malloc jitter, no free-list overhead



Single-Thread Engine

Predictable sequencing → no synchronization inside the order book

Each mechanism in this chapter is a direct implementation of one of these five principles.

SECTION 1

Why This Chapter Matters

The matching engine and the business case for low latency



Role in the Exchange

- Core component of every electronic trading exchange
- Accepts order requests from the Order Gateway Server
- Maintains the Limit Order Book and generates market data updates

Latency Is the Competitive Differentiator

- Exchanges with the best infrastructure attract the most order flow
- Lower latency → more participants → more revenue for the exchange
- Every microsecond saved translates directly to business advantage

The "Crossing" Concept

- A BUY order "crosses" a SELL when: $\text{buy_price} \geq \text{sell_price}$
- When prices cross → both orders execute (a trade occurs)
- Partial fills: leftover qty remains in the book
- The engine detects the cross and executes both sides atomically

THESIS: Memory layout + deterministic control flow

Order Flow



01

System Flow

How data moves between components
Client → Order Server → Matching Engine →
Market Data Publisher

02

Data Model

Structures that represent the order book
MEOrder · MEOrdersAtPrice · HashMaps · Memory
Pools

03

Execution Logic

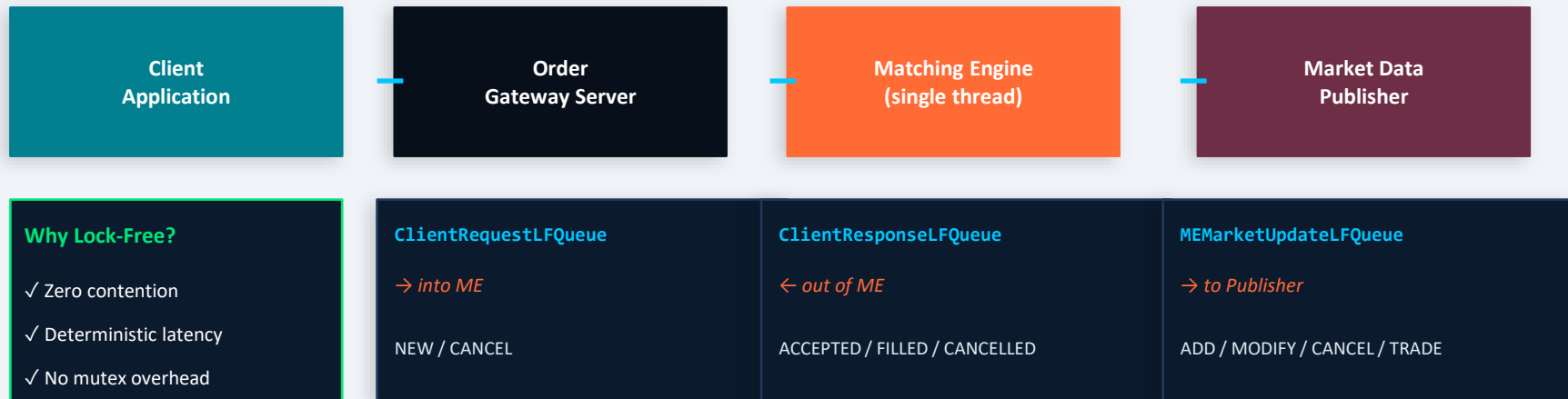
How orders are processed and matched
add() · cancel() · checkForMatch() · match()

SECTION 2

End-to-End Architecture

System flow, threading model, lock-free queues

02



THESIS: Memory layout + deterministic control flow

Single engine thread → no synchronization inside the order book. Lock-free queues at the boundary are the only concurrency mechanism.

Ask Book State Before Trade

Price=117 level:

MOID=1200

Prio=11 Qty=20

FULLY filled (20 of 25)

MOID=1400

Prio=13 Qty=10

PARTIALLY filled (5 of 10, 5 remain)

MOID=6200

Prio=1 Qty=200

Price 118 — not crossed

What the Engine Does

1

ACCEPTED response sent immediately

2

checkForMatch(): asks_by_price_ at 117

3

match(MOID=1200): fill 20, leaves_qty=5

4

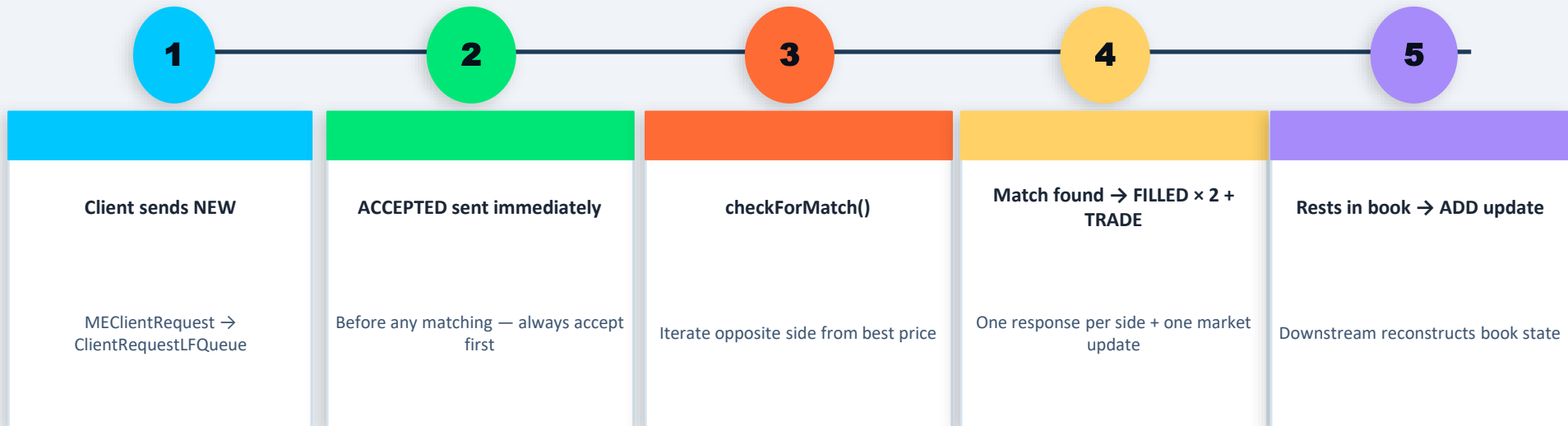
match(MOID=1400): fill 5, leaves_qty=0

5

leaves_qty=0 → order fully matched, done

Messages emitted:

FILLED (aggressor, 20 exec) · FILLED (MOID=1200, fully consumed) · TRADE · CANCEL(1200) | FILLED (5 exec) · FILLED (MOID=1400 partial) · TRADE · MODIFY(1400, qty=5)



SECTION 3

Order Book Mental Model

Build the structure incrementally: levels → orders → lookup maps

0

3

One Node Per Active Price

- Each distinct price with live orders = one MEOrdersAtPrice node
- Nodes doubly-linked from most → least aggressive on each side
- bids_by_price_ → head pointer (best bid, highest price)
- asks_by_price_ → head pointer (best ask, lowest price)

Why a Doubly-Linked List?

- Best price (head) is always $O(1)$ — critical for matching
- Price level removal is $O(1)$: splice out node, update prev/next
- Insert of a new price level: $O(k)$ walk to find position — but k is small in practice
- Doubly-linked (not singly) because removal needs to update prev pointer

Key Design Note

- The list only contains price levels that have at least one live order
- When the last order at a price is cancelled or filled, removeOrdersAtPrice() deletes the node
- No "empty" nodes persist — the list is compact

THESIS: Memory layout + deterministic control flow

Bid Side (sorted best → worst)

MEOrdersAtPrice
price=\$120

← bids_by_price_ (head)

MEOrdersAtPrice
price=\$119

prev ↑ / next ↓

MEOrdersAtPrice
price=\$118

prev ↑ / next ↓

MEOrdersAtPrice
price=\$117

prev ↑ / nullptr

One MEOrder Node Per Live Order

- MEOrdersAtPrice.first_me_order_ → head of the order chain
- Orders chained via prev_order_ / next_order_ in FIFO arrival order
- Earliest order (first-in) is at the head — executes first
- New orders append to the tail

Why Intrusive Pointers?

- prev_order_ and next_order_ are embedded inside MEOrder itself
- No separate list-node object → no extra allocation, no indirection
- To remove: set prev->next = next, next->prev = prev — O(1)
- This is the core memory layout insight for O(1) cancellation

Priority Field

- Each MEOrder carries a priority_ value (monotonically increasing)
- Higher priority_ = older order = earlier in the FIFO queue
- Market updates broadcast priority_ so downstream can reconstruct FIFO order
- getNextPriority(): returns prev_order_->priority_ + 1

THESIS: Memory layout + deterministic control flow

Orders at Price=117 (FIFO chain)

first_me_order_ →

MEOrder MOID=1200

prio=11 qty=20

HEAD — first to match



MEOrder MOID=1400

prio=13 qty=10

FIFO 2nd position



MEOrder MOID=1600

prio=15 qty=30

FIFO 3rd position

OrdersAtPriceHashMap: Price → Level

- `std::array<MEOrdersAtPrice*, ME_MAX_PRICE_LEVELS>`
- Index = `price % ME_MAX_PRICE_LEVELS` (direct modulo bucket)
- O(1) lookup: given a price, find its `MEOrdersAtPrice*` instantly
- No tree traversal, no pointer chasing — one array dereference

ClientOrderHashMap: ClientId → OrderId → Order

- `std::array<OrderHashMap, ME_MAX_NUM_CLIENTS>`
- Two-level: `cid_oid_to_order_[clientId][orderId]`
- O(1) lookup for cancel validation: is this order real?
- Also allows the engine to find order details given only client + order ID

Why Arrays Not `std::unordered_map`?

- `std::unordered_map`: bucket allocation, hash collision chains, heap nodes
- `std::array`: one contiguous block, index = identity, zero indirection
- Fixed capacity is a constraint — but it is the correct trade-off here

THESIS: Memory layout + deterministic control flow

How the Two Maps Relate

`price_orders_at_price_[ ]`

[0]	→ MEOrdersAtPrice price=117
[1]	→ MEOrdersAtPrice price=118
[2]	→ nullptr
[3]	→ nullptr
[4]	→ MEOrdersAtPrice price=117

⚡ `117 % MAX_LEVELS = [0]`

⚡ `118 % MAX_LEVELS = [1]`

Arrays + direct indexing = the "free lunch" of the data model

Four Interconnected Structures

`bids_by_price_ / asks_by_price_`

Head pointers into doubly-linked list of `MEOrdersAtPrice` nodes, sorted most → least aggressive.

`MEOrdersAtPrice`

One node per price level. `first_me_order_` → FIFO chain of `MEOrder`. Intrusive `prev/next` for price-level list. Allocated from `MemPool`.

`MEOrder`

Individual orders chained via intrusive `prev_order_ / next_order_` in FIFO arrival order. Allocated from `MemPool` — zero heap allocation.

Two Lookup Maps (`std::array`)

`OrdersAtPriceHashMap`: `price % MAX` → `MEOrdersAtPrice*`. `ClientOrderHashMap`: `[clientId][orderid]` → `MEOrder*`. Both $O(1)$.

How Each Operation Uses the Structures

`add()` `PriceHashMap` → find or create level → append `MEOrder` to FIFO tail → update `ClientOrderHashMap`

`cancel()` `ClientOrderHashMap` → find `MEOrder` → $O(1)$ pointer unlink → if last at level, remove level → return to pool

`checkForMatch()` `bids/asks head` → walk price levels → for each, walk FIFO chain → call `match()` per crossed order

`match()` Decrement qty → if fully consumed, `removeOrder()` → emit `FILLED + TRADE + CANCEL/MODIFY`

The Key Insight

Every structure is reachable via $O(1)$ index or pointer dereference. The hash maps provide random access; the linked lists provide ordered traversal. Together they make every hot-path operation near-constant time.

SECTION 4

Core Data Structures

MEOrder embeds linkage · MEOrdersAtPrice indexes levels

0

4

MEOrder struct —
annotated

```
struct MEOrder {
    TickerId  ticker_id_      = TickerId_INVALID;
    ClientId  client_id_     = ClientId_INVALID;

    OrderId   client_order_id_ // client-scoped
    OrderId   market_order_id_ // ← PUBLIC UNIQUE ID

    Side      side_          = Side::INVALID;
    Price     price_         = Price_INVALID;
    Qty       qty_           // remaining live qty
    Priority   priority_      // ← FIFO position

    MEOrder   *prev_order_    // ← INTRUSIVE prev
    MEOrder   *next_order_    // ← INTRUSIVE next
};

typedef std::array<MEOrder,
    ME_MAX_ORDER_IDS> OrderHashMap;
```

market_order_id_ → Public unique identifier

Generated by the engine. Globally unique across all clients. Used in all market data updates. client_order_id_ is client-scoped and NOT globally unique.

priority_ → FIFO queue position

Monotonically assigned at each price level. Broadcast in market updates so downstream consumers can reconstruct exact queue order without being connected to the exchange.

prev_order_ / next_order_ → Intrusive linkage

Embedded directly in MEOrder — no separate list-node allocation. Removal is O(1): two pointer writes. This is the key memory-layout insight for O(1) cancellation.

THESIS: Memory layout + deterministic control flow

MEOrdersAtPrice

struct -
annotated

```
struct MEOrdersAtPrice {  
    Side        side_      = Side::INVALID;  
    Price        price_    // ← THIS LEVEL'S PRICE  
    MEOrder *first_me_order_ // ← FIFO HEAD  
    MEOrdersAtPrice *prev_entry_ // ← INTRUSIVE prev  
    MEOrdersAtPrice *next_entry_ // ← INTRUSIVE next  
};
```

```
// Direct-index hash map  
typedef std::array<MEOrdersAtPrice*,  
    ME_MAX_PRICE_LEVELS> OrdersAtPriceHashMap;
```

```
//  $O(1)$  lookup  
auto priceToIndex(Price p) {  
    return p % ME_MAX_PRICE_LEVELS; }  

```

```
auto getOrdersAtPrice(Price p) {  
    return price_orders_at_price_  
        .at(priceToIndex(p));  
}
```

first_me_order_ → FIFO chain head

Points to the oldest (highest priority) order at this price level. checkForMatch() always starts here and walks via next_order_.

prev_entry_ / next_entry_ → Price-level list linkage

MEOrdersAtPrice itself is an intrusive node in the bids/asks sorted list. Same $O(1)$ unlink pattern as MEOrder — two pointer writes.

priceToIndex: price % MAX_LEVELS → $O(1)$ bucket access

Modulo converts any price into an array index. One array lookup replaces the $O(\log n)$ tree traversal of std::map. The constant factor advantage is enormous.

THESIS: Memory layout + deterministic control flow

vs. std::map: $O(\log n)$ traversal + heap nodes + pointer indirection. Array wins on all three counts.

SECTION 5

Message Structs & Queues

The communication layer: packed structs, typed responses, redundant updates

0

5

MEClientRequest

From: Order Server

To: Matching Engine

NEW • CANCEL

Key fields:

type_, client_id_, order_id_

side_, price_, qty_

#pragma pack → flat binary layout (network-ready)

MEClientResponse

From: Matching Engine

To: Order Server → Client

ACCEPTED • FILLED
CANCELLED • CANCEL_REJECTED

Key fields:

exec_qty_ = this fill only

leaves_qty_ = still live

#pragma pack → flat binary layout (network-ready)

MEMarketUpdate

From: Matching Engine

To: Market Data Publisher

ADD • MODIFY
CANCEL • TRADE

Key fields:

priority_ = FIFO position

order_id_ = market_order_id_

#pragma pack → flat binary layout (network-ready)

exec_qty_ is non-cumulative — each partial fill generates its own FILLED message with only that fill's quantity.

Design principle: TRADE + CANCEL/MODIFY always emitted together on a fill

A consumer that ignores TRADE messages can still maintain a perfectly accurate order book from CANCEL + MODIFY alone.

Consumer Type A · Full processing

Processes TRADE messages

- Can display real-time trade tape
- Can compute VWAP, traded volume, last price
- Receives all events including fills

Consumer Type B · Book-only

Ignores TRADE messages

- CANCEL removes passive order from book
- MODIFY updates passive order quantity
- Book remains 100% accurate regardless

Why is this an intentional design choice?

- Market data consumers have varying capabilities — not everyone can process every message type
- Redundancy costs a tiny amount of bandwidth but eliminates entire classes of downstream bugs
- This is a systems-design principle: intentional message redundancy to simplify consumer correctness guarantees

SECTION 6

Matching Engine Control Flow

Thread lifecycle, volatile caveat, run loop, and dispatch

0

6

Key Data Members

- `ticker_order_book_` → one `MEOrderBook` per instrument
- `incoming_requests_` → `ClientRequestLFQueue*` (in)
- `outgoing_ogw_responses_` → `ClientResponseLFQueue*` (out)
- `outgoing_md_updates_` → `MEMarketUpdateLFQueue*` (out)
- volatile bool `run_` → controls the run loop (see caveat below)

Thread Lifecycle

- `start()` → sets `run_=true` → spawns thread executing `run()`
- `stop()` → sets `run_=false` → loop condition fails → thread exits
- Deleted copy/move constructors prevent accidental object copies

⚠ Caveat: volatile in This Chapter

volatile is used here as part of a simplified threading model for the book. In production C++ concurrency, volatile alone does not guarantee memory ordering or visibility. Production systems would use `std::atomic<bool>` with appropriate memory order semantics.

C++

```
auto MatchingEngine::start() {
    run_ = true;
    ASSERT(
        Common::createAndStartThread(
            [this]() { run(); }
        ) != nullptr,
        "Failed to start ME thread."
    );
}

auto MatchingEngine::stop() {
    run_ = false;
}

// Production note: prefer
// std::atomic<bool> run_
// for correct memory semantics

// Boilerplate – prevent copies
MatchingEngine(const ME&) = delete;
ME& operator=(const ME&) = delete;
```

run() — Main Engine Loop

- Continuous busy-poll on incoming_requests_LFQueue
- LIKELY() hint: compiler optimizes for the "message is present" path
- processClientRequest() dispatches to the correct order book

processClientRequest() — Dispatch

- Reads type_ from MEClientRequest
- NEW → order_book->add(clientId, orderId, ticker, side, price, qty)
- CANCEL → order_book->cancel(clientId, orderId, ticker)
- Invalid type → FATAL assertion (should never happen)

Publishing Pattern

- sendClientResponse() and sendMarketUpdate() use identical pattern:
 1. getNextToWriteTo() — grab next free LFQueue slot
 2. std::move(data) into the slot — zero copy
 3. updateWriteIndex() — atomically advance write pointer
- No blocking. No allocation. Just a pointer advance.

C++

```
auto run() noexcept {
    while (run_) {
        const auto me_req =
            incoming_requests_
                ->getNextToRead();
        if (LIKELY(me_req)) {
            processClientRequest(me_req);
            incoming_requests_
                ->updateReadIndex();
        }
    }
}

// Publish pattern
auto sendClientResponse(
    const MEClientResponse& r) {
    auto next = outgoing_ogw_
        ->getNextToWriteTo(); // 1
    *next = std::move(r);    // 2
    outgoing_ogw_
        ->updateWriteIndex(); // 3
}
```

SECTION 7

Core Order Book Operations

★ *add · cancel · checkForMatch · match — the hot path*

1

Generate market_order_id_

2

Send ACCEPTED — always, immediately

3

checkForMatch() → returns leaves_qty

4

If leaves_qty > 0 → rest in book



Memory pool: order_pool_.allocate() — zero heap allocation in hot path

C++

```
// Step 1
const auto new_moid =
    generateNewMarketOrderId();

// Step 2: accept immediately
client_response_ = {ACCEPTED,
    client_id, ticker_id,
    client_order_id, new_moid,
    side, price, qty};
matching_engine_>sendClientResponse(...);

// Step 3: check for match
const auto leaves_qty =
    checkForMatch(..., new_moid);

// Step 4: rest in book
if (LIKELY(leaves_qty)) {
    const auto priority =
        getNextPriority(price);
    auto order = order_pool_.allocate(...);
    addOrder(order);
    // send ADD market update
}
```

When add() creates a new price level:

- 1 Insert into price_orders_at_price_ hash map (O(1) — direct array index)
- 2 Walk the bids/asks doubly-linked list from the best price to find the correct sorted insertion point
- 3 Splice the new MEOrdersAtPrice node into the list by updating prev/next pointers
- 4 If the new level is more aggressive than the current best, update bids_by_price_ or asks_by_price_ head pointer

Complexity: O(k) where k = number of active price levels

In practice k is small — real order books rarely have more than a few dozen active price levels on each side.

```
C++ (simplified)
// Step 1: hash map insert
price_orders_at_price_
    .at(priceToIndex(price))
    = new_orders_at_price;

// Step 2: walk to find position
auto target = best_orders_by_price;
bool add_after = /* price comparison */;
while(add_after &&
      target != best_orders) {
    // advance through levels
    target = target->next_entry_;
}

// Step 3: splice into list
new_oap->prev_ = target;
target->next_->prev_ = new_oap;
new_oap->next_ = target->next_;
target->next_ = new_oap;

// Step 4: update best if needed
if (new_is_more_aggressive)
    bids/asks_by_price_ = new_oap;
```



This is the only operation that walks a linked list to find position. All other hot-path operations are O(1). An alternative implementation could walk the hash map instead — trade-offs depend on level count vs. price spacing.

Validate

client_id in range? AND cid_oid_to_order_[clientId][orderId] ≠ nullptr?

✗ INVALID

→ Send CANCEL_REJECTED response

✓ VALID

→ Send CANCELLED response

→ Send CANCEL market update

→ removeOrder(): O(1) unlink

→ If last at level: removeOrdersAtPrice()

→ Return MEOrder to order_pool_

💡 removeOrder is O(1): two pointer assignments. order_pool_.deallocate() is O(1): return slot to pool.

C++

```
auto MEOrderBook::cancel(...) {
    auto is_cancellable =
        (client_id <
         cid_oid_to_order_.size());
    if (LIKELY(is_cancellable)) {
        exchange_order =
            cid_oid_to_order_
                .at(client_id)
                .at(order_id);
        is_cancellable =
            (exchange_order != nullptr);
    }

    if (UNLIKELY(!is_cancellable)) {
        // CANCEL_REJECTED path
    } else {
        removeOrder(exchange_order); // O(1)
        order_pool_.deallocate(...); // O(1)
    }
    matching_engine_->sendClientResponse(...);
}
```

Recall Slide 6: BUY 25 @ 117 → asks at 117 (qty=20, qty=10). This is checkForMatch() in action.

- 1 Start at best price (`asks_by_price_ head`)
- 2 For each price level, iterate FIFO chain (`first_me_order_ → next_order_`)
- 3 Call `match()` for each order whose price $\leq$ aggressor price
- 4 Stop: `leaves_qty==0`, prices no longer cross, or book side empty
- 5 Return remaining `leaves_qty` to `add()`

Complexity: $O(k)$ over crossed orders. In practice k is very small — usually 0 or 1. For BUY 25@117: $k=2$ (fills MOID 1200 and 1400 partially).

C++

```
auto checkForMatch(...) {
    auto leaves_qty = qty;

    if (side == Side::BUY) {
        while (leaves_qty && asks_by_price_) {
            const auto ask_itr =
                asks_by_price_
                    ->first_me_order_;

            // price no longer crosses?
            if (LIKELY(price < ask_itr->price_))
                break;

            match(..., ask_itr,
                &leaves_qty);
        }
    }
    // mirror for SELL side
    return leaves_qty;
}
```

1

Crossing Decision

Already made by checkForMatch() — match() is only called when crossing is confirmed. No re-check.

2

Fill Calculation

fill_qty = min(leaves_qty, order->qty_). Updates both leaves_qty and order->qty_ in place.

3

State Mutation

Decrement two counters. No allocation. No copy. Pure pointer arithmetic.

4

Four Messages Emitted

FILLED (aggressor) · FILLED (passive) · TRADE · CANCEL (fully filled) or MODIFY (partial)

C++

```
// Steps 2 & 3
const auto fill_qty = std::min(
    *leaves_qty, order->qty_);
*leaves_qty -= fill_qty;
order->qty_ -= fill_qty;

// Step 4a: aggressor FILLED
client_response_ = {FILLED,
    client_id, ..., fill_qty,
    *leaves_qty};

// Step 4b: passive FILLED
client_response_ = {FILLED,
    order->client_id, ...,
    fill_qty, order->qty_};

// Step 4c: TRADE update
market_update_ = {TRADE, ..., fill_qty};

// Step 4d: CANCEL or MODIFY
if (!order->qty_) { // fully consumed
    removeOrder(order); // CANCEL
} else {
    // partial → MODIFY
}
```

Operation	What It Does	Complexity
<code>add()</code> resting order	lookup + pool allocate + linked insert	Near $O(1)$
<code>cancel()</code> valid order	hash lookup + pointer unlink + pool free	$O(1)$
best price access	<code>bids_by_price_ / asks_by_price_ head ptr</code>	$O(1)$
<code>checkForMatch()</code>	iterate crossed orders + call <code>match()</code>	$O(k)$, <code>k = fills</code>
<code>addOrdersAtPrice()</code>	walk sorted price list to insert new level	$O(k)$, <code>k = levels deep</code>

The Real Insight: Asymptotics Are Necessary But Not Sufficient

- ▶ Constant factors matter more than O-notation when `k` is always small (1–5 fills is typical)
- ▶ Cache locality matters: `std::array` is contiguous — cache-friendly in a way `std::list` cannot be
- ▶ Allocation behavior matters: `pool.allocate()` is ~3 instructions vs. `malloc` which may take microseconds
- ▶ This is why the design replaces every allocating, branching, or traversing operation with a non-allocating, branch-free, direct-access alternative

SECTION 8

Design Decisions & Summary

Trade-offs · Production scope · Three principles to remember

08

```
std::list<MEOrder>
```

Each node heap-allocated separately. Linked via external pointers. Removal requires an iterator — no $O(1)$ access by OrderId.

✓ **MEOrder intrusive**

Intrusive doubly-linked list inside MEOrder. Removal via pointer stored in MEOrder. $O(1)$ from any access path.

```
std::map<Price, Level>
```

Red-black tree. $O(\log n)$ lookup. Each node heap-allocated. Pointer chasing at every tree level. Cache-unfriendly.

✓ **Array + modulo**

`std::array` indexed by `price % MAX_LEVELS`. $O(1)$ lookup. One contiguous block. Cache-friendly direct access.

```
std::unordered_map<OrderId,  
Order>
```

Bucket allocation. Hash collision chains. Load factor resizing. Heap nodes per entry.

✓ **Array direct index**

`std::array<MEOrder, MAX_ORDER_IDS>`. OrderId is the direct index. One dereference. Zero allocation.

```
new MEOrder / delete
```

`malloc/free`: lock on the global allocator, potential fragmentation, non-deterministic latency.

✓ **Memory pool**

`MemPool<MEOrder>`: pre-allocated slab. `allocate()` / `deallocate()` = ~ 3 instructions each. Fully deterministic.

#pragma pack on message structs only

Packed for network transmission. NOT global — internal structs keep natural alignment for CPU efficiency.

INVALID sentinel on every type

Safe default; uninitialized values are detectable. Better than null pointers for POD types.

Array-backed "hash maps"

Fixed capacity, no resizing, no heap nodes. Constraint: you must know MAX capacity up front.

Powers-of-two constants

Enables bit-shift potential. Effect is negligible on modern CPUs — but zero cost to maintain.

TRADE + CANCEL/MODIFY redundancy

Extra events cost bandwidth but guarantee book correctness for all consumer types (see Slide 14).

Memory pools for MEOrder + MEOrdersAtPrice

Eliminates ALL dynamic allocation from the hot path. Fixed slab size = known memory footprint.

✓ What This Chapter Optimizes

- ▶ Single-instrument matching logic — clean, teachable, complete
- ▶ Memory discipline — no heap allocation in the hot path
- ▶ Deterministic hot path — lock-free queues, direct indexing
- ▶ Correct FIFO price-time priority execution
- ▶ $O(1)$ order lifecycle: add, cancel, match
- ▶ Thread isolation — all shared state via LFQueues only

⚠ What Real Production Systems Also Need

- ▶ Multi-symbol partitioning — parallel order books per instrument
- ▶ Recovery and persistence — crash-safe order state
- ▶ Sequencing guarantees — sequence numbers on all updates
- ▶ Pre-trade risk checks — position limits, fat-finger prevention
- ▶ Network stack integration — kernel bypass (e.g. Solarflare)
- ▶ Monitoring, alerting, and graceful degradation

Three Ideas to Remember

01

Low latency starts with data layout

Before you optimize an algorithm, ask: where is the data? Is it contiguous? Is it hot in cache? Does accessing it require an allocation?

This chapter answers every one of those questions in the same way: use an array, embed the linkage, pre-allocate the pool.

02

Determinism often matters more than average speed

A system that is fast on average but occasionally slow is often worse than a system that is consistently fast.

Lock-free queues, memory pools, and a single-thread engine all exist to eliminate the variance, not just improve the mean.

03

The matching engine is fast because it removes avoidable work

There is no malloc in the hot path. There is no tree traversal. There is no lock contention. There is no external list-node allocation.

The design is not clever. It is disciplined. Every mechanism removes a category of overhead that the generic solution would have left in.